

AI Agent Workflow Builder

A mini n8n for chaining AI agent steps — organizations build multi-step workflows (`llm_call` , `http_request` , `db_write` , `notify` , `conditional_branch` , `approval_gate`), start them four ways (manual, webhook, scheduled, database event), and every action is checked against two independent permission layers. Built on **nhost** (Postgres + Hasura + Auth + Storage + Functions) and **Next.js**.

See [WRITEUP.md](#) for the schema/permissions/pause-resume design write-up.

Repo layout

<code>nhost/</code>	nhost project: config, SQL migrations, Hasura metadata (tables, relationships, permissions, Actions, cron/event triggers) – git-trackable, applied
<code>functions/</code>	nhost Functions (TypeScript) – the Action handlers and the run execution
<code>graphql/</code>	documented GraphQL operations (query/mutation/subscription) the frontend
<code>scripts/</code>	<code>seed.mjs</code> – sets up the Final Task demo scenario end to end
<code>web/</code>	Next.js (App Router) frontend

Prerequisites

- Node.js 18+, Docker Desktop (for local nhost)
- **nhost CLI** — `brew install nhost/tap/nhost`
- A free Groq API key (console.groq.com) for `llm_call` steps — or see **Stubbing without a key** below

Local setup

1. Start the backend:

```
nhost up
```

This starts Postgres, Hasura, Auth, Storage, and Functions in Docker, and applies every migration + the full Hasura metadata (tables, relationships, both permission layers, Actions, cron/event triggers) already committed in `nhost/` . On first run it prints the local URLs (Hasura console, GraphQL, Auth, etc.) and an admin secret in `.secrets` (gitignored, auto-generated).

2. Add your Groq API key: Edit `.secrets` (created by `nhost up`) and set:

```
GROQ_API_KEY = 'your-key-here'
```

Then `nhost up` again to pick it up (it's wired in via `nhost.toml` 's `[[global.environment]]` , which injects it into the Functions container).

3. Seed the Final Task demo scenario:

```
HASURA_ADMIN_SECRET=$(grep '^HASURA_GRAPHQL_ADMIN_SECRET' .secrets | cut -d'=
node scripts/seed.mjs
```

This creates two organizations (Org A with an owner/editor/viewer, Org B with an owner), and a fully-built demo workflow in Org A (`llm_call` → `conditional_branch` → `http_request` → `approval_gate` , with both a manual and a webhook trigger). It prints all the login credentials, IDs, and a ready-to-run `curl` command for the webhook trigger.

4. Run the frontend:

```
cd web
cp .env.example .env.local # already points at local nhost URLs by default
npm install
npm run dev
```

Open <http://localhost:3000> (or whatever port it picks — grafana/other local services sometimes hold 3000, in which case use `npm run dev -- --port 3001`).

Stubbing without a Groq key

If you don't want to grab a Groq key, `llm_call` steps will fail with a clear `GROQ_API_KEY is not configured` error rather than silently doing nothing — the retry/failure-handling path (3 attempts, exponential backoff, `step_runs.error` populated, run marked `failed`) is exactly what you'd see and is itself part of what's being demonstrated. To fully stub it instead, edit `functions/lib/groq.ts` 's `callGroq` to return a canned `{ content, model }` after an artificial `await new Promise(r => setTimeout(r, 800))` delay — the disclosed-stub approach the assignment allows.

Notify step (Slack/email)

`notify` steps are stubbed: the run executor inserts into `notifications` (status `pending`) and moves on immediately; a Hasura Event Trigger on that table's inserts calls `functions/notify-dispatcher.ts` , which currently just logs the message and marks it `sent` . To wire up a real Slack incoming webhook: set `SLACK_WEBHOOK_URL` the same way as `GROQ_API_KEY` (add it to `.secrets` + `[[global.environment]]` in `nhost.toml`), no code changes needed — `notify-dispatcher.ts` already checks for it.

Deployment

- **Backend:** `nhost login` , link this project to an nhost Cloud project (`nhost link`), then push config/migrations/metadata (`nhost config apply` / the Cloud dashboard's Git integration). Set `GROQ_API_KEY` as a Cloud secret the same way as locally.

- **Frontend**: deployed to Vercel, pointed at the Cloud project's Auth/GraphQL URLs via the same three `NEXT_PUBLIC_NHOST_*` env vars as `web/.env.example`.

Live URLs:

- Hosted app: <https://web-pearl-ten-22.vercel.app>
- nhost Cloud project: `rbjdbrkxebgtgftiznht` (region `ap-south-1`) — auto-deploys on every push to `main` (migrations, metadata, and Functions all apply automatically)
- GitHub repo: https://github.com/MmohammedH/vocallabs_assignment

To seed the Final Task demo scenario against the **cloud** backend (rather than local), grab the project's admin secret from the nhost dashboard (Settings → Environment Variables → reveal `NHOST_ADMIN_SECRET`) and:

```
HASURA_ADMIN_SECRET=<revealed value> \  
AUTH_URL=https://rbjdbrkxebgtgftiznht.auth.ap-south-1.nhost.run/v1 \  
GRAPHQL_URL=https://rbjdbrkxebgtgftiznht.graphql.ap-south-1.nhost.run/v1 \  
node scripts/seed.mjs
```

A note on scope

This was built under real time pressure (the assignment says as much is expected). Where I traded breadth for correctness of the core integration, it's called out in code comments and in `WRITEUP.md` — e.g. the executor runs synchronously inside the Action's HTTP request rather than on a queue, the scheduled trigger is one static cron poller at 1-minute granularity rather than per-workflow dynamic cron registration, and `db_write` writes to an allow-listed sink table rather than arbitrary SQL. None of these affect the two permission layers, the Action handler's own role checks, or cross-org isolation — those are the parts the Final Task actually grades, and they're the parts most heavily tested (see the browser E2E and isolation testing described in `WRITEUP.md`).